

Repo Health Check 软件工程文档集

本文档集用于课程验收 项目答辩 后续维护和开源协作 文档按照软件工程常见交付件组织 并结合 Repo Health Check 当前实现状态编写

文档目录

文档	说明
01-需求规格说明书 (. /01-requirements-specification.md)	项目背景 目标用户 功能需求 非功能需求 验收标准
02-可行性分析报告 (. /02-feasibility-study.md)	技术 经济 进度 风险与法律合规可行性
03-概要设计说明书 (. /03-architecture-design.md)	系统架构 模块划分 部署视图 关键流程
04-详细设计说明书 (. /04-detailed-design.md)	核心模块 算法策略 异常处理和关键数据结构
05-数据库设计说明书 (. /05-database-design.md)	持久化对象 表结构 缓存策略和数据生命周期
06-API 接口设计说明书 (. /06-api-design.md)	REST API 请求响应格式 错误码和安全要求
07-测试计划与测试用例 (. /07-test-plan.md)	测试范围 测试策略 用例矩阵 验收测试
08-部署与运维手册 (. /08-deployment-operations.md)	本地启动 CI/CD GitHub/GitLink 常见故障
09-项目管理计划 (. /09-project-management-plan.md)	迭代计划 任务拆解 协作流程 配置管理
10-风险与质量保证计划 (. /10-risk-quality-plan.md)	风险识别 质量目标 安全策略 回归机制
11-用户使用手册 (. /11-user-manual.md)	面向普通用户的功能说明和操作步骤
12-需求追踪矩阵 (. /12-traceability-matrix.md)	需求 设计 实现 测试和验收之间的映射

项目概述

Repo Health Check 是一个面向开源项目的仓库健康度体检工具 用户输入 GitHub 仓库 URL 后 系统会克隆并分析仓库 从代码质量 测试覆盖 架构健康 文档完整 依赖安全和工程规范六个维度输出健康度评分 雷达图 问题列表 AI 诊断建议 README Badge 导出报告 排行榜和仓库 PK 对比结果

当前稳定版本

- 稳定标签 [v2.1.0](#)
- 推荐运行方式 双击项目根目录的 [Start-RepoHealth.bat](#)
- 前端地址 <http://127.0.0.1:5174>
- 后端地址 <http://127.0.0.1:8002>

需求规格说明书

1. 引言

1.1 编写目的

本文档定义 Repo Health Check 的业务目标 用户角色 功能需求 非功能需求 约束条件和验收标准为开发 测试 答辩和后续维护提供统一依据

1.2 项目背景

开源项目数量庞大 但普通用户 学生开发者和项目维护者往往难以快速判断一个仓库是否健康 传统判断方式依赖人工查看 README 测试目录 依赖文件 Issue 状态和代码结构 效率低且主观性强 Repo Health Check 希望通过自动化分析和可视化报告 降低开源项目评估门槛

1.3 目标用户

用户角色	典型需求
开源项目维护者	快速发现项目质量短板 获得改进建议
课程评审/教师	通过报告 CI 文档和演示判断项目工程化程度
学生开发者	学习开源项目质量指标和软件工程实践
普通 GitHub 用户	判断一个仓库是否值得使用 学习或贡献

2. 总体目标

- 输入 GitHub 仓库 URL 后自动生成健康度报告
- 从六个维度量化仓库质量
- 输出可视化报告 Badge AI 诊断和导出文件
- 支持仓库排行榜 投票 历史趋势和双仓库对比
- 通过 OpenSpec CI/CD 测试和文档体现完整软件工程实践

3. 功能需求

FR-01 仓库分析

系统应支持用户输入合法 GitHub 仓库 URL 并对仓库进行克隆 扫描 聚合评分

验收标准

- 非法 URL 显示输入错误
- 合法 URL 能进入报告页
- 分析成功后返回总分 等级 六维分数和问题列表

FR-02 六维健康度评分

系统应从以下维度分析仓库

维度	内容
代码质量	复杂度 重复/坏味道 语言分布 静态检查
测试覆盖	测试文件 测试框架 CI 测试命令
架构健康	目录结构 模块边界 循环依赖 规模风险
文档完整	README 贡献指南 变更日志 安全策略
依赖安全	依赖文件 漏洞扫描 锁文件 过期风险
工程规范	CI/CD 格式化 Lint EditorConfig 环境样例

FR-03 报告展示

报告页应展示

- 总健康分与等级
- 六维雷达图
- 维度分数条
- 语言分布饼图
- AI 诊断建议
- 问题列表
- Badge 嵌入代码
- 分享链接

FR-04 AI 诊断

系统应支持两级诊断

- 无 API Key 使用本地规则自动生成建议
- 有 DeepSeek/OpenAI Key 调用 LLM 生成更详细建议

验收标准

- 用户不配置 Key 也能获得诊断

- 配置 Key 后不持久化保存到后端
- 诊断项包含严重程度 预估工时 置信度和是否需要人工确认

FR-05 缓存感知分析

系统应在 30 分钟缓存有效期内直接返回已有结果 避免重复克隆和分析

验收标准

- 同一仓库短时间重复访问优先命中缓存
- 错误缓存不会长期阻塞重新分析

FR-06 导出与分享

系统应支持

- HTML 报告导出
- 浏览器打印为 PDF
- 分享卡片生成
- README Badge 生成

FR-07 仓库对比

系统应支持输入两个不同仓库 URL 展示双方总分 维度对比 雷达图 PK Arena 和海报生成

验收标准

- 不允许同一仓库自我对比
- 从报告页跳转时可预填仓库 A
- A / VS / B 布局稳定居中

FR-08 排行榜与投票

系统应记录分析历史 支持排行榜展示 分页 趋势标记和用户投票

FR-09 用户认证

系统应支持 GitHub OAuth 登录 用于扩展个性化使用场景

FR-10 工程化与文档

系统应包含

- GitHub Actions CI
- GitLink DevOps 配置说明

- OpenSpec capability 归档
- CONTRIBUTING SECURITY CHANGELOG
- 软件工程文档集

4. 非功能需求

类型	需求
性能	普通中小型仓库 30-180 秒内完成分析 缓存命中应秒级返回
可用性	前端提供进度提示 错误提示 输入校验和一键启动脚本
安全性	防止 XSS Session 密钥硬编码 OAuth state 缺失 跨域误配
可靠性	后台任务有锁保护 临时目录最终清理 错误缓存可恢复
可维护性	模块按 routes services analyzer components pages 划分
可测试性	后端 pytest 前端 Vitest CI 执行 lint/test/build
可扩展性	可继续接入更多分析维度 数据库 CI 平台和语言分析器

5. 约束条件

1. GitHub 仓库需要可通过网络克隆
2. 大型仓库可能超时 需要提示用户稍后重试
3. LLM 诊断依赖用户提供或环境配置 API Key
4. Windows 本地运行需安装 Python Node.js 和 Git
5. GitHub/GitLink CI 依赖平台网络和构建镜像

6. 验收标准

1. 双击 `Start-RepoHealth.bat` 可启动前后端
2. 首页高级 UI Dock 导航和报告页正常展示
3. 输入公开 GitHub 仓库可生成报告
4. Compare Leaderboard About 页面功能可访问
5. `npm run lint npm test -- --run npm run build` 通过
6. 后端 pytest 和 E2E 脚本可执行
7. GitHub 上存在稳定版本标签 `v2.1.0`

可行性分析报告

1. 技术可行性

1.1 技术栈成熟度

层级	技术	可行性说明
后端	Python / FastAPI / uvicorn	生态成熟 适合快速构建 REST API
分析	radon / lizard / bandit / pip-audit	覆盖复杂度 代码规模 安全漏洞和依赖风险
前端	React / TypeScript / Vite / ECharts	支持复杂交互和可视化报告
存储	SQLite / JSON 兼容策略	轻量 零配置 适合课程项目和单机部署
CI/CD	GitHub Actions / GitLink DevOps	可自动验证 lint test build
规范	OpenSpec	适合能力变更记录和需求追踪

结论 技术栈均为成熟开源方案 项目技术可行

1.2 关键技术风险

风险	影响	应对
GitHub 网络不稳定	克隆失败或耗时过长	支持代理 超时 错误提示 缓存
大仓库分析慢	用户等待时间长	异步任务 状态轮询 缓存命中
LLM API 不可用	AI 诊断失败	无 Key 本地规则诊断降级
多语言分析复杂	分析准确度不一致	先覆盖 Python/JS/TS 逐步扩展

2. 经济可行性

项目主要基于开源技术构建 不依赖付费基础设施 可选成本来自

- LLM API 调用费用
- 云服务器或部署平台费用
- GitHub Actions/GitLink 构建资源

课程实践和本地演示场景可零成本运行 经济可行

3. 进度可行性

项目采用 capability 迭代方式推进 已经完成 cap-01 到 cap-20 的能力归档 覆盖从项目初始化 分析引擎 报告 AI 前端改版 安全加固 CI/CD 到报告体验增强的完整链条

进度结论 当前版本已经达到课程答辩和演示要求

4. 操作可行性

用户操作路径简单

1. 双击一键启动脚本
2. 输入 GitHub 仓库 URL
3. 等待进度条完成
4. 查看报告 导出 分享或对比

系统对无 Key 缓存命中 网络失败 输入错误均有提示 操作可行

5. 法律与合规可行性

- 项目遵循开源依赖许可证使用方式
- 不保存用户输入的 API Key 到后端
- GitHub OAuth 仅用于登录扩展 不强制使用
- 公开仓库分析不涉及私有代码泄露

结论 在只分析公开仓库的前提下合规风险较低

6. 综合结论

Repo Health Check 在技术 经济 进度 操作和合规方面均具备可行性 适合作为开源软件课程实践项目 也具备后续扩展为真实开源质量评估工具的潜力

概要设计说明书

1. 架构目标

系统采用前后端分离架构 目标是

- 前端负责输入 可视化 交互和导出体验
- 后端负责克隆仓库 分析 缓存 持久化和 API
- 分析器按维度插件化 便于扩展
- 数据层轻量化 适合本地和课程部署

2. 总体架构

```
flowchart LR
    User["用户"] --> FE["React 前端"]
    FE --> API["FastAPI REST API"]
    API --> Clone["仓库克隆服务"]
    API --> Cache["结果缓存"]
    API --> Storage["SQLite/历史存储"]
    Clone --> Repo["临时仓库目录"]
    Repo --> Analyzer["六维分析器"]
    Analyzer --> AI["本地规则/LLM 诊断"]
    Analyzer --> API
    API --> FE
```

3. 模块划分

3.1 前端模块

模块	职责
HomePage	输入 GitHub URL API Key 配置 推荐仓库
ReportPage	报告展示 进度条 导出 分享 Badge
ComparePage	双仓库对比 PK Arena 海报生成
LeaderboardPage	排行榜 分页 投票
AboutPage	项目介绍 capability 轨迹 技术栈
components	RadarChart ScoreBar LanguagePieChart Dock GlassCard 等
utils	PDF 打印 分享卡片 PK 海报等前端工具

3.2 后端模块

模块	职责
routes/analyze.py	分析任务创建 缓存命中 状态轮询
routes/badge.py	SVG Badge 生成
routes/export.py	HTML 报告导出
routes/compare.py	双仓库并行分析
routes/leaderboard.py	排行榜查询
routes/vote.py	投票接口
services/clone.py	Git 仓库克隆 代理降级
services/cache.py	TTL 缓存 深拷贝保护
services/storage.py	历史记录 投票和持久化
services/session.py	Session 签名密钥
analyzer/	六维分析器和聚合器
ai/diagnose.py	LLM/本地规则诊断

4. 部署视图

flowchart TB
Browser["浏览器 5174"] --> Backend["FastAPI 8002"]
Backend --> Data["data/*.db"]
Backend --> Temp["临时 clone 目录"]
Backend --> GitHub["GitHub 仓库"]
Backend --> LLM["DeepSeek/OpenAI 可选"]

5. 关键流程

5.1 分析流程

1. 用户输入 GitHub URL
2. 前端调用 `POST /api/analyze`
3. 后端检查缓存
4. 缓存命中则直接返回
5. 未命中则创建异步任务并返回 `task_id`
6. 前端轮询 `GET /api/analyze/status`
7. 后端克隆仓库并运行六维分析

8. 聚合评分 生成 AI 诊断 保存缓存和历史
9. 前端展示报告

5.2 对比流程

1. 用户输入仓库 A 和仓库 B
2. 前端校验两个 URL 均合法且不同
3. 后端并行分析两个仓库
4. 前端展示总分对比 雷达图 维度差异和 PK Arena

5.3 导出流程

1. 前端通过 hash 获取导出 URL
2. 后端生成 HTML 报告
3. 用户可下载 HTML 或用浏览器打印为 PDF
4. 分享卡片和 PK 海报由前端 Canvas 生成

6. 安全设计

- HTML/SVG 输出统一转义 防止 XSS
- Session 未配置密钥时拒绝启动
- OAuth 使用 state 防 CSRF
- CORS 仅允许配置的前端来源
- API Key 不写入后端持久化存储
- 异步任务集合通过锁保护

7. 扩展设计

后续可扩展

- 接入 PostgreSQL
- 支持 GitLab/GitLink 仓库
- 增加 Lighthouse Issue 响应速度 PR 合并速度等维度
- 支持组织级仓库批量分析
- 生成更完整的项目健康度趋势预测

详细设计说明书

1. 分析聚合器设计

聚合器负责调用各维度分析器 并将结果统一转成前端可展示的数据结构

核心输出

```
interface AnalysisData {  
  repo_url: string;  
  health_score: number;  
  badge_level: string;  
  badge_color: string;  
  badge_description: string;  
  dimensions: AnalysisDimension[];  
  ai_diagnosis: AIDiagnosisItem[];  
  analyzed_at: string;  
}
```

评分策略

1. 每个维度输出 0-100 分
2. 聚合器按维度平均或权重计算总分
3. 总分映射到 Badge 等级
4. 诊断建议按严重程度排序

2. 六维分析器设计

2.1 代码质量分析器

输入 仓库路径

处理

- 识别 Python JavaScript TypeScript Go 等语言文件
- 使用 radon/lizard 统计复杂度
- 统计语言分布 代码规模和高复杂函数

输出

- score
- issues
- details.language_breakdown

2.2 测试覆盖分析器

处理

- 检测 `tests/ __tests__/ *.test.* *.spec.*`
- 检测 `pytest Vitest Jest` 等配置
- 检测 CI 中是否执行测试

2.3 架构健康分析器

处理

- 检查目录分层是否清晰
- 识别过大文件和模块
- 检查循环依赖或高耦合风险

2.4 文档完整性分析器

处理

- 检查 `README CONTRIBUTING SECURITY CHANGelog LICENSE`
- 检查 `.env.example` 是否覆盖实际环境变量
- 检查 `API/部署/测试说明`

2.5 依赖安全分析器

处理

- 检查 `requirements package.json lock` 文件
- 使用 `bandit/pip-audit` 等工具识别安全问题
- 对缺少锁文件或依赖过旧给出建议

2.6 工程规范分析器

处理

- 检查 `CI/CD lint format` 测试脚本
- 检查 `EditorConfig Prettier Black ESLint` 等配置
- 检查 `Docker` 启动脚本和部署说明

3. 缓存设计

缓存键 规范化后的仓库 URL hash

缓存行为

- 成功结果保存 30 分钟

- 缓存命中直接返回
- 错误结果不长期阻塞 允许重新分析
- 返回缓存结果时使用深拷贝 避免外部修改内部状态

4. 异步任务设计

后端使用 `asyncio.create_task` 创建后台分析任务

任务状态

- `pending` 任务正在分析
- `completed` 缓存中存在成功结果
- `failed` 缓存中存在错误结果
- `not_found` 任务不存在或已过期

竞态保护

- `_pending_tasks` 通过 `threading.Lock` 保护

5. AI 诊断设计

诊断输入

- 六维分析结果
- 仓库 URL
- 可选 API Key

诊断输出

```
interface AIDiagnosisItem {  
    advice: string;  
    severity: 'high' | 'medium' | 'low';  
    estimated_hours: number;  
    confidence: number;  
    need_human_review: boolean;  
}
```

降级策略

- 用户未配置 Key 本地规则诊断
- LLM 请求失败 回退本地规则
- 缓存无诊断且用户请求诊断 补充 AI 诊断并更新缓存

6. 前端交互设计

6.1 首页

首页使用高级视觉组件

- Iridescence WebGL 背景
- CurvedLoop 标题
- GlassCard 输入区
- Dock 底部导航

6.2 报告页

状态

- loading 分步进度条
- success 报告内容
- error 错误卡片

核心交互

- 导出 HTML
- 打印 PDF
- 生成分享卡片
- 复制 Badge
- 跳转对比页并预填仓库 A

6.3 对比页

校验

- A/B 都必须是 GitHub URL
- A/B 不能相同

展示

- A / VS / B 三列布局
- PK Arena
- 雷达图和维度差异
- PK 海报生成

7. 异常处理

场景	处理方式
URL 非法	前端提示错误

场景	处理方式
克隆失败	后端返回失败结果 前端展示错误
分析超时	返回 timeout 并清理临时目录
LLM 失败	回退本地诊断
Badge 数据缺失	返回灰色未知 Badge
导出弹窗被拦截	前端提示允许弹窗

数据库设计说明书

1. 数据设计目标

Repo Health Check 的持久化数据主要用于

- 保存仓库历史分析记录
- 支持排行榜
- 保存投票数据
- 支持趋势展示和重复分析对比

系统优先采用 SQLite 适合本地启动 课程演示和轻量部署

2. 主要实体

2.1 AnalysisHistory

表示一次仓库分析摘要

字段	类型	说明
id	integer	主键
repo_url	text	规范化后的仓库 URL
health_score	real	健康分
badge_level	text	等级
badge_color	text	Badge 颜色
analyzed_at	text/datetime	分析时间

2.2 Vote

表示用户对仓库的投票统计

字段	类型	说明
id	integer	主键
repo_url	text	仓库 URL
vote_count	integer	投票数量
updated_at	text/datetime	更新时间

2.3 CacheRecord

当前缓存主要在内存中维护 也可扩展为数据库表

字段	类型	说明
repo_hash	text	仓库 hash
repo_url	text	仓库 URL
result_json	text	完整分析结果
expires_at	datetime	过期时间

3. 数据关系

```
erDiagram
    ANALYSIS_HISTORY {
        int id
        string repo_url
        float health_score
        string badge_level
        string badge_color
        datetime analyzed_at
    }
    VOTE {
        int id
        string repo_url
        int vote_count
        datetime updated_at
    }
    ANALYSIS_HISTORY ||--o{ VOTE : "repo_url"
```

4. 索引设计

表	索引	目的
analysis_history	repo_url	查询单仓库历史趋势
analysis_history	health_score	排行榜排序
analysis_history	analyzed_at	获取最新记录
vote	repo_url	投票查询和更新

5. 数据生命周期

- 1. 用户分析仓库后保存历史摘要
- 2. 完整报告优先进入缓存
- 3. 缓存 30 分钟后过期

4. 历史记录长期保留 用于排行榜和趋势
5. 临时 clone 目录在分析结束后删除

6. 一致性策略

- 分析完成后再写入历史记录
- 排行榜以仓库最新或最高得分记录作为展示依据
- 投票接口需要防止短时间重复投票
- 多进程部署时建议使用 SQLite 或 PostgreSQL 避免 JSON 文件并发写问题

7. 后续扩展

- 引入 Alembic 管理数据库迁移
- 将完整报告结果持久化
- 增加用户表和用户收藏表
- 支持组织级仓库批量分析记录

API 接口设计说明书

1. 通用响应格式

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

code	说明
0	成功
1	异步任务处理中
-1	失败

2. POST /api/analyze

启动或获取仓库分析

请求

```
{
  "repo_url": "https://github.com/vuejs/vue",
  "force_sync": false,
  "skip_ai": false,
  "ai_api_key": "optional"
}
```

成功返回

```
{
  "code": 0,
  "message": "success (cached)",
  "data": {
    "repo_url": "...",
    "health_score": 85,
    "badge_level": "A",
    "dimensions": [],
    "ai_diagnosis": []
  }
}
```

异步返回

```
{
  "code": 1,
  "message": "analyzing",
  "data": {
    "task_id": "abc123",
    "repo_url": "https://github.com/vuejs/vue",
  }
}
```

```
    "status": "pending"
  }
}
```

3. GET /api/analyze/status

查询异步分析状态

参数

参数	必填	说明
task_id	是	任务 ID
repo_url	否	仓库 URL 用于精确查询

返回状态

- pending
- completed
- failed
- not_found

4. GET /api/badge/{hash}

生成仓库健康度 SVG Badge

安全要求

- label/value 必须 HTML 转义
- 输出 Content-Type 为 SVG

5. GET /api/export/{hash}

导出 HTML 报告

安全要求

- 所有动态字段必须 HTML 转义
- 报告可被浏览器打印为 PDF

6. POST /api/compare

对比两个仓库

请求

```
{
  "repo_a": "https://github.com/vuejs/vue",
  "repo_b": "https://github.com/facebook/react",
  "skip_ai": false
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "repo_a": {},
    "repo_b": {}
  }
}
```

7. GET /api/leaderboard

获取排行榜

建议参数

参数	说明
page	页码
page_size	每页数量

8. POST /api/vote

为仓库投票

请求

```
{
  "repo_url": "https://github.com/vuejs/vue"
}
```

限制

- 应支持冷却时间 防止刷票

9. GET /api/history/{repo}

获取某仓库历史趋势

响应

```
{
  "code": 0,
  "data": [
```

```
{
  "analyzed_at": "2026-06-25T10:00:00",
  "health_score": 82,
  "badge_level": "B"
}
]
```

10. GET /health

健康检查

响应

```
{
  "status": "ok"
}
```

测试计划与测试用例

1. 测试目标

验证 Repo Health Check 在主要功能 异常场景 安全边界 前端交互和工程化流程中的正确性和稳定性

2. 测试范围

范围	内容
后端单元测试	分析器 缓存 导出 Badge API 排行榜
前端单元测试	核心组件 Context 键盘可访问性
集成测试	前后端 API 串联 分析流程
E2E 测试	本地自分析 健康检查 报告生成
安全测试	XSS 转义 Session 密钥 CORS
CI 测试	GitHub Actions / GitLink DevOps

3. 测试环境

项目	配置
操作系统	Windows 10/11 或 Linux
Python	3.11+
Node.js	20+
后端	FastAPI + uvicorn
前端	Vite dev server
数据库	SQLite

4. 测试命令

后端

```
python -m pytest backend/tests -q
python scripts/e2e_test.py
```

前端

```
cd frontend
npm run lint
npm test -- --run
npm run build
```

5. 功能测试用例

编号	用例	步骤	预期
TC-FR-01	首页输入合法仓库	输入 <code>https://github.com/vuejs/vue</code> 并提交	跳转报告页并开始分析
TC-FR-02	首页输入非法 URL	输入普通字符串	显示输入错误
TC-FR-03	报告生成	等待分析完成	展示总分 雷达图 AI 建议
TC-FR-04	缓存命中	短时间再次打开同仓库报告	快速返回缓存结果
TC-FR-05	Badge 复制	点击复制 Badge	剪贴板写入 Markdown
TC-FR-06	HTML 导出	点击导出报告	下载 HTML 文件
TC-FR-07	PDF 导出	点击导出 PDF	打开打印窗口
TC-FR-08	分享卡片	点击生成分享卡片	下载 PNG
TC-FR-09	对比不同仓库	输入 A/B 两个仓库	展示 A / VS / B 对比
TC-FR-10	对比相同仓库	A/B 输入相同 URL	显示错误提示
TC-FR-11	排行榜投票	点击投票按钮	投票数增加或进入冷却提示
TC-FR-12	About 页面	打开 <code>/about</code>	展示 cap-01 到 cap-20

6. 安全测试用例

编号	场景	输入	预期
TC-SEC-01	Badge XSS	label/value 包含 <code><script></code>	SVG 中被转义
TC-SEC-02	HTML 导出 XSS	advice 包含 HTML 标签	导出报告中被转义
TC-SEC-03	Session 密钥缺失	不配置 SESSION_SECRET	服务拒绝生产启动
TC-SEC-04	CORS	非白名单来源请求	被拒绝
TC-SEC-05	OAuth state	缺失或错误 state	登录失败

7. 性能测试用例

编号	场景	指标
TC-PERF-01	缓存命中	1 秒内返回
TC-PERF-02	中小型仓库分析	180 秒内完成
TC-PERF-03	Compare 并行分析	耗时接近单仓库较慢者 而非两者相加

8. 回归测试清单

每次发布前必须验证

- 首页高级 UI 未退化
- 报告页导出和 AI 诊断正常
- Compare A / VS / B 布局正常
- About cap 数量正确
- `npm run lint` 通过
- `npm test -- --run` 通过
- `npm run build` 通过
- 后端 `pytest` 通过

部署与运维手册

1. 本地一键启动

Windows 推荐方式

Start-RepoHealth.bat

启动后

- 后端 <http://127.0.0.1:8002>
- 前端 <http://127.0.0.1:5174>
- 健康检查 <http://127.0.0.1:8002/health>

2. 手动启动

后端

```
pip install -r backend/requirements.txt
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8002 --reload
```

前端

```
cd frontend
npm install
npm run dev -- --host 127.0.0.1 --port 5174
```

3. 环境变量

变量	说明
SESSION_SECRET	Session 签名密钥 生产环境必填
CORS_ORIGINS	允许访问后端的前端来源
FRONTEND_URL	OAuth 登录后跳转地址
DEEPSEEK_API_KEY	可选 LLM 诊断
OPENAI_API_KEY	可选 LLM 诊断备用
GIT_HTTP_PROXY	可选 Git 克隆代理
TRUST_PROXY_HEADERS	是否信任反向代理头
ALLOW_NPX_INSTALL	是否允许自动安装 npx 工具

4. GitHub Actions

GitHub CI 建议包含

1. Python lint
2. 后端 pytest
3. E2E 自测
4. 前端 lint
5. 前端 Vitest
6. 前端 build

5. GitLink DevOps

推荐图形流水线

GitLink trigger -> 开始 -> backend shell -> frontend shell -> 结束

backend shell

```
sh devops/gitlink/backend-ci.sh
```

frontend shell

```
sh devops/gitlink/frontend-ci.sh
```

6. 发布流程

1. 确认工作区干净
2. 执行测试和构建
3. 提交代码
4. 创建版本标签
5. 推送到 GitHub
6. 创建 GitHub Release

示例

```
git commit -m "release: stabilize v2.1.0"
git tag -a v2.1.0 -m "v2.1.0 stable UI and UX release"
git push origin master
git push origin v2.1.0
```

7. 常见故障

问题	原因	解决
前端无法连接后端	端口/CORS 不一致	确认前端 5174 后端 8002
浏览器显示旧 UI	旧 Vite 服务占用端口	关闭旧终端 重新启动
克隆 GitHub 失败	网络或代理问题	配置 <code>GIT_HTTP_PROXY</code>
分析超时	仓库过大	使用缓存或分析较小仓库
PDF 无法导出	浏览器拦截弹窗	允许弹窗后重试

8. 运维检查

发布前检查

```
python -m pytest backend/tests -q
cd frontend
npm run lint
npm test -- --run
npm run build
```

项目管理计划

1. 管理目标

项目管理目标是保障 Repo Health Check 在有限课程周期内完成可运行 可演示 可测试 可追踪的开源软件实践项目

2. 过程模型

项目采用迭代式开发与 OpenSpec 规范驱动开发结合

- 1. 提出 capability
- 2. 编写 proposal/design/tasks/spec
- 3. 实现功能
- 4. 测试验证
- 5. 归档到 [openspec/changes/archive/](#)

3. 里程碑

阶段	目标	产出
M1 项目初始化	前后端骨架 OpenSpec 配置	cap-01
M2 分析引擎	六维分析器	cap-02
M3 报告 API	分析 Badge 缓存 导出	cap-03
M4 AI 诊断	LLM 和本地规则诊断	cap-04
M5 前端体验	高级 UI 雷达图 报告页	cap-05/cap-16
M6 工程化	CI/CD 测试 部署 文档	cap-06/cap-19
M7 产品增强	PDF 分享卡片 排行榜 PK	cap-20
M8 稳定发布	标签 Release 答辩材料	v2.1.0

4. 任务拆解

类型	示例任务
后端	API 分析器 缓存 存储 安全加固
前端	首页 报告页 对比页 排行榜 About 页

类型	示例任务
AI	Prompt 结构化输出 本地规则降级
工程化	CI GitLink 脚本 测试 文档
产品	分享卡片 PDF Badge PK 海报

5. 协作规范

分支建议

feature/*
fix/*
docs/*
release/*

提交信息建议

feat: add report export
fix: prevent badge xss
docs: add software engineering documents
release: stabilize v2.1.0

6. 配置管理

关键配置

- `.env.example` 环境变量模板
- `frontend/.env` 前端 API 地址
- `.github/workflows/ci.yml` GitHub CI
- `devops/gitlink/*.sh` GitLink DevOps 脚本
- `openspec/` 规范和能力归档

7. 质量门禁

每次合并前至少满足

- 代码能启动
- 前端 lint/test/build 通过
- 后端测试通过
- 不破坏稳定 UI
- 更新对应文档或 OpenSpec 归档

风险与质量保证计划

1. 质量目标

目标	度量
功能正确	核心流程可运行 主要测试通过
前端稳定	首页 报告 对比 排行榜 About 不回退
安全可靠	XSS Session CORS 任务竞态有防护
易维护	模块清晰 文档完整 OpenSpec 可追踪
可发布	有 tag release notes CI/CD

2. 风险清单

风险	概率	影响	应对
UI 被后续功能覆盖破坏	中	高	使用 v2.1.0 tag 锁定稳定版 功能按小块移植
GitHub 克隆失败	高	中	代理配置 超时 错误提示
大仓库分析慢	中	中	异步任务 进度条 缓存
LLM API 不可用	中	低	本地规则诊断降级
XSS 注入	低	高	HTML/SVG 转义
多进程数据不一致	中	中	SQLite 替代 JSON 文件
CI 配置不兼容平台	中	中	GitHub Actions 与 GitLink 分别维护

3. 质量保证活动

3.1 代码审查

重点审查

- 用户输入
- HTML/SVG 输出
- 文件系统操作
- 临时目录清理

- 缓存和异步任务并发

3.2 自动化测试

- 后端 pytest
- 前端 Vitest
- 前端 lint/build
- E2E 自测脚本

3.3 回归测试

关键 UI 回归

- 首页高级视觉
- Report 页面导出和 AI 诊断
- Compare 页面 A / VS / B 布局
- About cap 数量

4. 发布质量门禁

发布前必须完成

```
python -m pytest backend/tests -q
cd frontend
npm run lint
npm test -- --run
npm run build
```

5. 回退策略

当前稳定版本

```
git checkout v2.1.0
```

创建恢复分支

```
git checkout -b restore-v2.1.0 v2.1.0
```


用户使用手册

1. 启动系统

Windows 用户双击

Start-RepoHealth.bat

启动成功后浏览器打开

<http://127.0.0.1:5174>

2. 分析仓库

1. 在首页输入 GitHub 仓库 URL
2. 点击“开始体检”
3. 等待进度条完成
4. 查看报告

示例

<https://github.com/vuejs/vue>

3. 查看报告

报告包含

- 健康总分
- 等级 Badge
- 六维雷达图
- 语言分布
- 各维度详情
- AI 诊断建议
- 问题列表
- Badge 嵌入代码

4. 导出报告

报告页支持

- 导出 HTML
- 导出 PDF
- 生成分享卡片
- 复制分享链接
- 复制 README Badge

5. 配置 AI Key

首页可配置 DeepSeek API Key 配置后

- Key 仅保存在本地浏览器
- 分析完成后自动使用 LLM 诊断
- 不配置 Key 也会使用本地规则诊断

6. 对比仓库

1. 点击 Dock 中的对比入口
2. 输入仓库 A 和仓库 B
3. 点击开始对比
4. 查看 A / VS / B 总分 雷达图 维度差异和 PK Arena
5. 可生成 PK 海报

注意 A 和 B 不能是同一个仓库

7. 查看排行榜

排行榜展示已分析仓库的健康分 等级 趋势和投票数 用户可以为仓库点赞

8. 常见问题

无法连接后端

确认后端运行在

`http://127.0.0.1:8002`

并访问

`http://127.0.0.1:8002/health`

页面还是旧样式

关闭旧的前端终端 重新运行 `Start-RepoHealth.bat` 然后按 `Ctrl + F5` 强制刷新

分析一直等待

可能是仓库过大或 GitHub 网络不稳定 可配置 `GIT_HTTP_PROXY` 或换一个较小仓库测试

需求追踪矩阵

1. 功能需求追踪

需求 ID	需求	设计/模块	实现位置	测试/验收
FR-01	仓库分析	分析流程 异步任务	backend/routes/analyze.py, backend/analyzer/	backend/tests/test_api.py, E2E
FR-02	六维评分	六维分析器	backend/analyzer/*	backend/tests/test_aggregator.py
FR-03	报告展示	ReportPage	frontend/src/pages/ReportPage.tsx	前端 build 人工验收
FR-04	AI 诊断	ai/diagnose	backend/ai/diagnose.py	backend/tests/test_ai_diagnose.py
FR-05	缓存感知	Cache service	backend/services/cache.py	backend/tests/test_cache.py
FR-06	导出分享	export/reportAssets	backend/routes/export.py, frontend/src/Utils/reportAssets.ts	backend/tests/test_export.py
FR-07	仓库对比	ComparePage/compare route	backend/routes/compare.py, frontend/src/pages/ComparePage.tsx	人工验收 build
FR-08	排行榜投票	leaderboard/vote	backend/routes/leaderboard.py, backend/routes/vote.py	backend/tests/test_leaderboard.py
FR-09	用户认证	OAuth/session	backend/routes/auth.py, backend/services/session.py	手工登录测试
FR-10	工程文档	docs/OpenSpec	docs/, openspec/	文档审查

2. 非功能需求追踪

非功能需求	实现策略	验证方式
性能	缓存 异步任务 Compare 并行	E2E 人工计时
安全	XSS 转义 Session 强密钥 CORS OAuth state	安全测试用例

非功能需求	实现策略	验证方式
可用性	进度条 错误提示 一键启动	人工验收
可维护性	模块化目录 OpenSpec 软件工程文档	代码审查
可测试性	pytest Vitest CI	自动化测试
可扩展性	分析器分层 API 分层 SQLite	架构审查

3. OpenSpec Capability 映射

Capability	覆盖需求
cap-01 project-init	FR-10
cap-02 analysis-engine	FR-01, FR-02
cap-03 api-report	FR-01, FR-03, FR-06
cap-04 ai-diagnosis	FR-04
cap-05 frontend-polish	FR-03
cap-06 deploy-e2e	FR-10
cap-07 polish-final	FR-07, FR-08
cap-08 async-optimize	FR-01, NFR 性能
cap-14 proxy-auto-degrade	NFR 可靠性
cap-16 react-bits-visual-redesign	FR-03, NFR 可用性
cap-18 xss-security-fix	NFR 安全
cap-19 security-architecture-hardening	NFR 安全 可维护性
cap-20 report-leaderboard-ux	FR-03, FR-04, FR-06, FR-08